



ISO 9001: 2015
SOCOTEC SCP000722Q

REPUBLIC OF THE PHILIPPINES
BICOL UNIVERSITY
POLANGUI
COMPUTER STUDIES DEPARTMENT
Polangui, Albay



IT 112 Web Systems and Technologies

BSIT-2B 2nd Semester

QuickServe Ordering System

GONZALES, SAMUEL L.

MOENDEM, ROEL B.

TAMBOBONG, NICKA D.

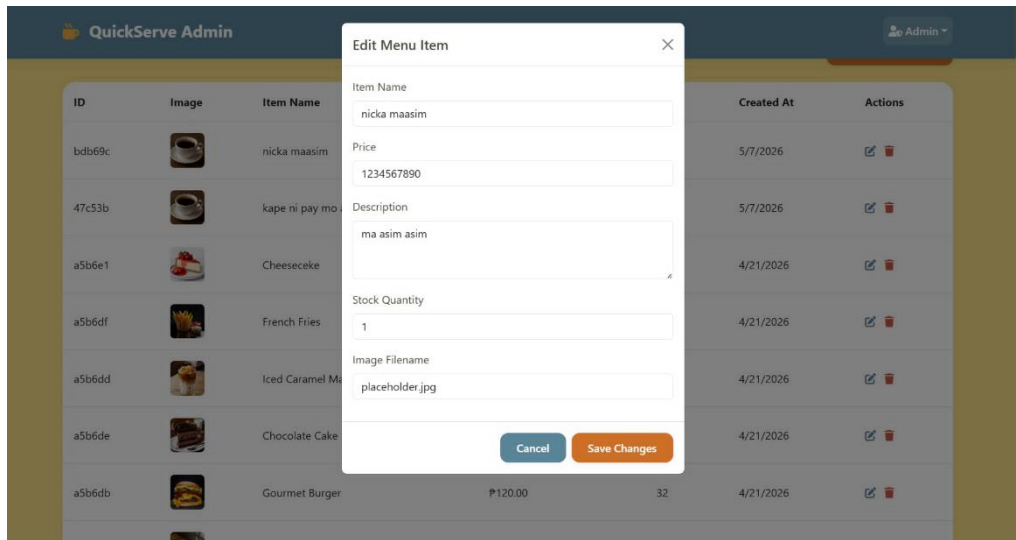
SADIA, MAICEL JOY

PIELAGO, AEDAN JADEN

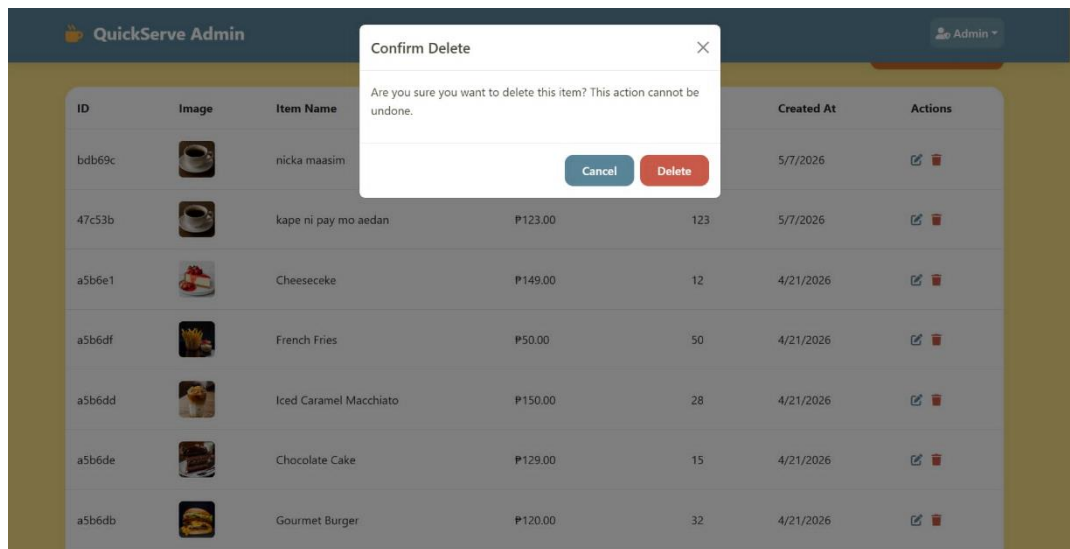
Submitted to:
Guillermo V. Red, Jr.

May 9, 2026

working edit modal or page



deletion confirmation and database updates



JavaScript snippets for edit and delete

ID	Image	Item Name	Price	Stock	Created At	Actions
bdb69c		nicka maasim	₱1234567890.00	1	5/7/2026	Edit Delete
a5b6e1		Cheeseceke	₱149.00	12	4/21/2026	Edit Delete
a5b6df		French Fries	₱50.00	50	4/21/2026	Edit Delete
a5b6dd		Iced Caramel Macchiato	₱150.00	28	4/21/2026	Edit Delete
a5b6de		Chocolate Cake	₱129.00	15	4/21/2026	Edit Delete
a5b6db		Gourmet Burger	₱120.00	32	4/21/2026	Edit Delete
a5b6dc		Italian Pizza	₱399.00	18	4/21/2026	Edit Delete

Backend route and controller snippets

```
443 async function editProduct(id) {
444   try {
445     const res = await fetch('http://localhost:3000/api/products/${id}', { headers: adminHeaders() });
446     const p = await res.json();
447     document.getElementById('editProductId').value = p._id;
448     document.getElementById('editProductName').value = p.name;
449     document.getElementById('editProductPrice').value = p.price;
450     document.getElementById('editProductDesc').value = p.description || '';
451     document.getElementById('editProductStock').value = p.stock || 0;
452     document.getElementById('editProductImage').value = p.image || '';
453     new bootstrap.Modal(document.getElementById('editProductModal')).show();
454   } catch (err) {
455     alert('Failed to load product details');
456   }
457 }
```

```
702 document.getElementById('confirmDeleteBtn')?.addEventListener('click', async function() {
703   if (!currentDeleteId) return;
704   let url = '';
705   if (currentDeleteType === 'product') url = 'http://localhost:3000/api/products/${currentDeleteId}';
706   else if (currentDeleteType === 'review') url = 'http://localhost:3000/api/reviews/${currentDeleteId}';
707   else if (currentDeleteType === 'user') url = 'http://localhost:3000/api/users/${currentDeleteId}';
708
709   try {
710     const res = await fetch(url, { method: 'DELETE', headers: adminHeaders() });
711     if (res.ok) {
712       showNotification(`${currentDeleteType.charAt(0).toUpperCase() + currentDeleteType.slice(1)} deleted successfully!`);
713       bootstrap.Modal.getInstance(document.getElementById('deleteModal')).hide();
714       currentDeleteId = null;
715       if (currentDeleteType === 'product') loadProducts();
716       else if (currentDeleteType === 'review') loadReviews();
717       else if (currentDeleteType === 'user') loadUsers();
718     } else {
719       const d = await res.json();
720       showNotification(d.message || 'Failed to delete', 'error');
721     }
722   } catch (err) {
723     showNotification('Connection error', 'error');
724   }
725 });
```

```
109 // PUT /api/products/:id (admin)
110 router.put('/:id', protect, admin, async (req, res) => {
111   try {
112     const { name, price, description, stock, image, imageData, category, available } = req.body;
113     const update = {};
114     if (name !== undefined) update.name = name.trim();
115     if (price !== undefined) update.price = parseFloat(price);
116     if (description !== undefined) update.description = description.trim();
117     if (stock !== undefined) update.stock = parseInt(stock);
118     if (image !== undefined) update.image = image || 'placeholder.jpg';
119     if (imageData !== undefined) update.imageData = imageData || null;
120     if (category !== undefined) update.category = category;
121     if (available !== undefined) update.available = available;
122
123     const product = await Product.findByIdAndUpdate(req.params.id, update, { new: true, runValidators: true });
124     if (!product) return res.status(404).json({ success: false, message: 'Item not found' });
125     res.json({ success: true, product });
126   } catch (error) {
127     res.status(400).json({ success: false, message: error.message });
128   }
129 });
```

```
131 // DELETE /api/products/:id (admin)
132 router.delete('/:id', protect, admin, async (req, res) => {
133   try {
134     const product = await Product.findByIdAndDelete(req.params.id);
135     if (!product) return res.status(404).json({ success: false, message: 'Item not found' });
136     res.json({ success: true, message: 'Menu item deleted successfully' });
137   } catch (error) {
138     res.status(500).json({ success: false, message: error.message });
139   }
140 });
141
142 module.exports = router;
143
```

Github commits logs

Commits

main

All users All time

Commits on May 5, 2026

ReadMe	maicelsadia-png authored 4 days ago	Verified	fb901d5		
ReadMe	maicelsadia-png authored 4 days ago	Verified	6b759b1		
Add files via upload	samuelgonzales37266136-tech authored 4 days ago	Verified	7b97abd		
Updated	Aladen011 committed 4 days ago		1953f57		

Short reflections

Frontend

During Phase 6, I managed to add editing and deleting operations on the admin panel and customer orders. The key issues were making sure that the edit forms would fill up with existing information and the delete prompt would have the right ID number. Eventually, I added put requests for changing product information and the order status, and delete requests for deleting products, reviews, and users.

Backend

As the Backend Developer, I worked on implementing the PUT and DELETE endpoints and updating the controllers for editing and removing data. This experience helped me become more familiar with handling backend logic and testing whether the changes worked correctly with the frontend and database. It also improved my problem-solving skills since I had to fix errors and make sure everything was properly connected and synchronized.

Database

As a database manager, verifying document updates and deletions was not easy for me at first. I followed the instructions on Bulms, but I encountered some problems and could not make it work properly, so I used AI to help me figure out the steps and understand the errors.

Github

As the GitHub Manager of our group, I was responsible for handling branch merging and keeping our commits synchronized. This role helped me improve my organization and teamwork skills because I had to make sure everyone's work was updated properly in the repository. I also learned how important communication is when resolving merge conflicts and managing different contributions from group members. Through this experience, I became more confident in using GitHub and understanding collaborative software development..

Documentation

As the Documentation Officer for **QuickServe**, I was responsible for preparing the visuals and notes included in our lab report. I organized screenshots, explanations, and other details to make our documentation clear and easy to understand. Even though completing everything on time was challenging, the experience helped me become more patient, organized, and responsible when working with a team.